

SWE 4739

Embedded Software Development

Lecture 7

Continuous and Discrete Dynamics

Sabrina Islam

Lecturer, CSE, IUT

Contact: +8801832239897

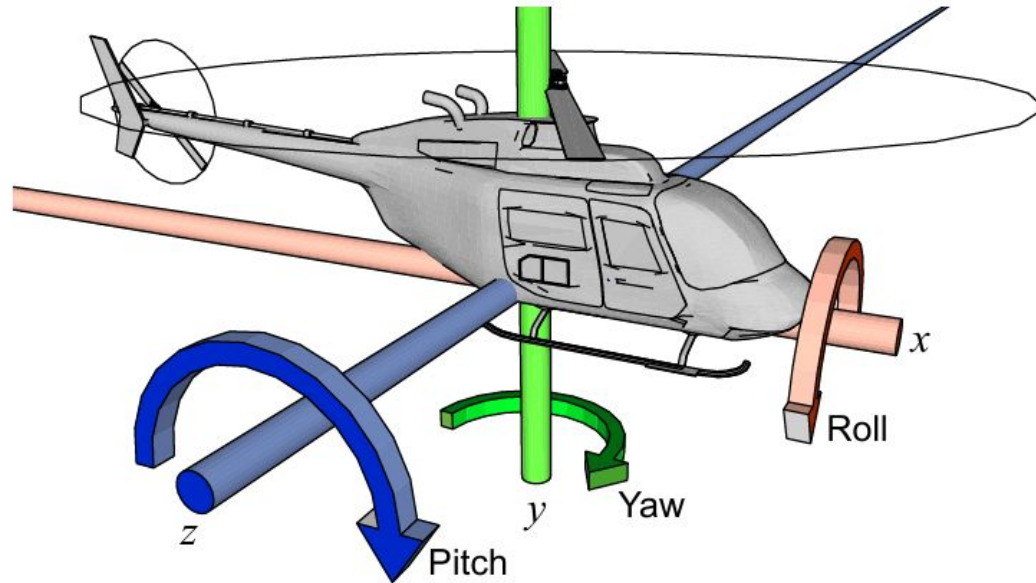
E-mail: sabrinaislam22@iut-dhaka.edu

Newtonian Mechanics

Motion in space of physical objects can be represented with six degrees of freedom.

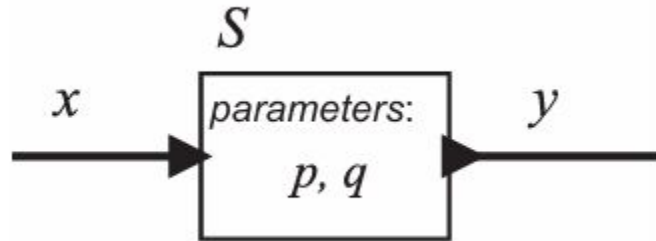
- Three of these represent position in three dimensional space, and three represent orientation in space.
- Three axes, x, y, and z
- **Roll θ_x** is an angle of rotation around the x axis
- **Yaw θ_y** is the rotation around the y axis
- **Pitch θ_z** is rotation around the z axis
- The position of an object in space, therefore, is represented by six functions of the form $f : \mathbb{R} \rightarrow \mathbb{R}$, known as **continuous-time signals**.

Newtonian Mechanics



Actor Models

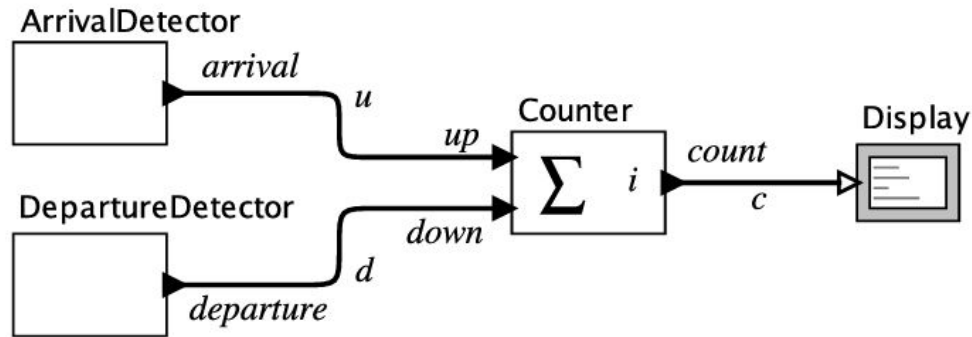
A continuous-time system (one that operates on continuous-time signals) maybe modeled by a box with an input port and a output port as follows:
(where the input signal x and the output signal y are functions of the form $x: \mathbb{R} \rightarrow \mathbb{R}$, $y: \mathbb{R} \rightarrow \mathbb{R}$)



Discrete Systems

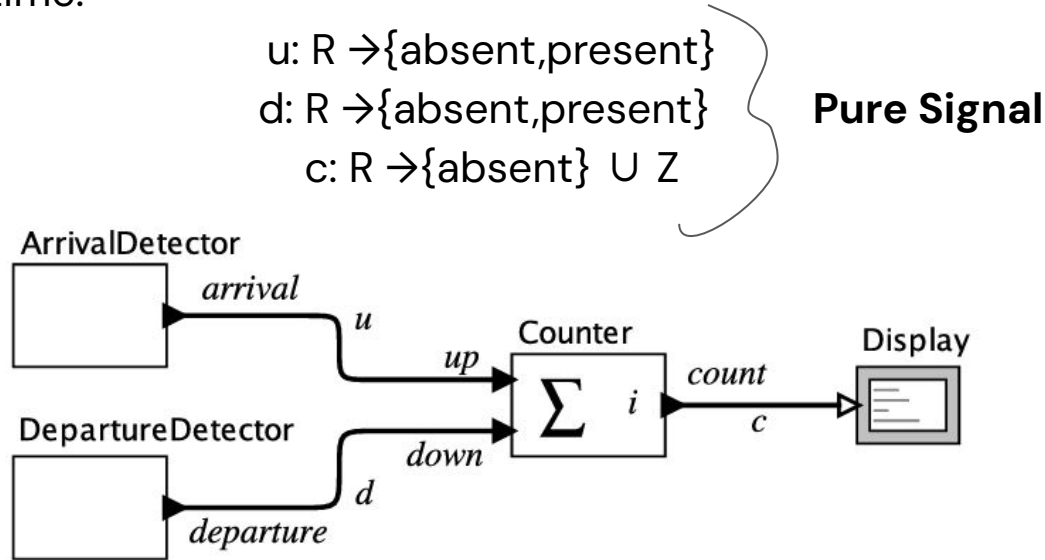
A discrete system operates in a sequence of discrete steps and is said to have discrete dynamics.

Example: Consider a system that counts the number of cars that enter and leave a parking garage in order to keep track of how many cars are in the garage at any time.



Discrete Systems

Example: Consider a system that counts the number of cars that enter and leave a parking garage in order to keep track of how many cars are in the garage at any time.



The Notion of State

- The state of a system is its condition at a particular point in time.
- The state affects how the system reacts to inputs.
- The state of the actor at a time t is the value of the integral of the input signal up to time t .
- For the Counter used in the previous example, the state $s(t)$ at time t is an integer, so **States $\subset \mathbb{Z}$** .
 - States = $\{0, 1, 2, \dots, M\}$

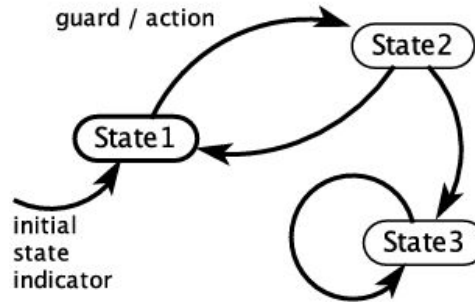
Finite-State Machines

- A state machine is a model of a system with discrete dynamics that at each reaction maps valuations of the inputs to valuations of the outputs, where the map may depend on its current state.
- A finite-state machine (FSM) is a state machine where the set States of possible states is finite.
- States = {State1,State2,State3}

Finite-State Machines

Transitions

- Transitions between states govern the discrete dynamics of the state machine and the mapping of input valuations to output valuations.
- The **guard** determines whether the transition may be taken on a reaction.
- The **action** specifies what outputs are produced on each reaction.



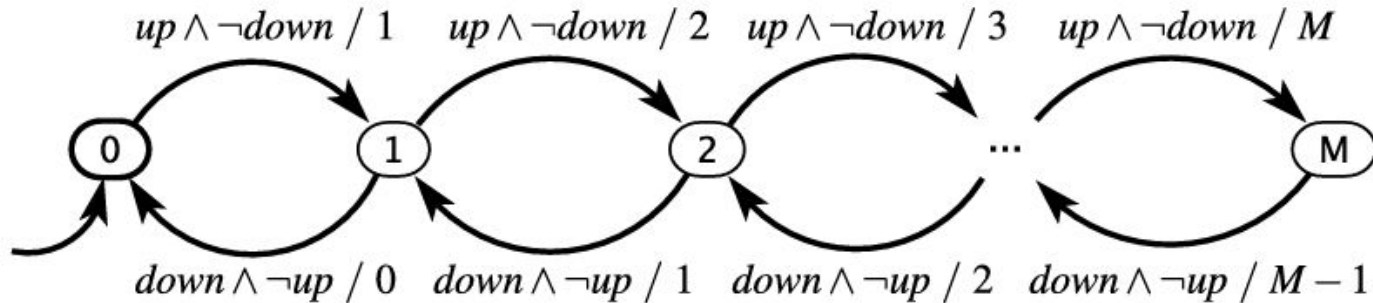
Finite-State Machines

FSM model for the **garage counter**

- The set of states is $\text{States} = \{0, 1, 2, \dots, M\}$.

inputs: $up, down$: pure

output: $count : \{0, \dots, M\}$

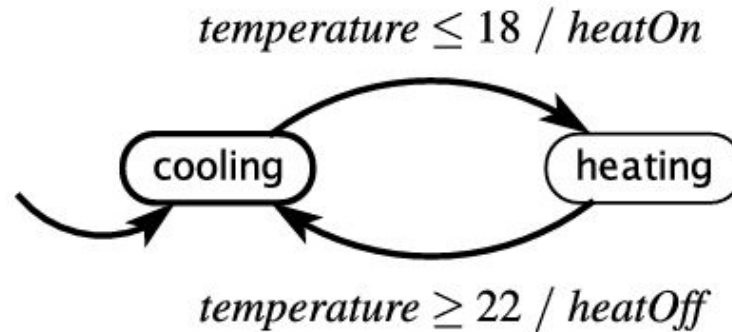


Finite-State Machines

FSM model for a **thermostat**

input: *temperature* : $\mathbb{R}$

outputs: *heatOn*, *heatOff* : pure



Finite-State Machines

- A finite-state machine is a five-tuple:
 - (States, Inputs, Outputs, update, initialState)
 - **States** is a finite set of states
 - **Inputs** is a set of input valuations
 - **Outputs** is a set of output valuations
 - **update** : States $\times$ Inputs $\rightarrow$ States $\times$ Outputs
 - **initialState** is the initial state

Finite-State Machines

Update Functions

$$\begin{aligned} \text{States} &= \{0, 1, \dots, M\} \\ \text{Inputs} &= (\{up, down\} \rightarrow \{present, absent\}) \\ \text{Outputs} &= (\{count\} \rightarrow \{0, 1, \dots, M, absent\}) \\ \text{initialState} &= 0 \end{aligned}$$

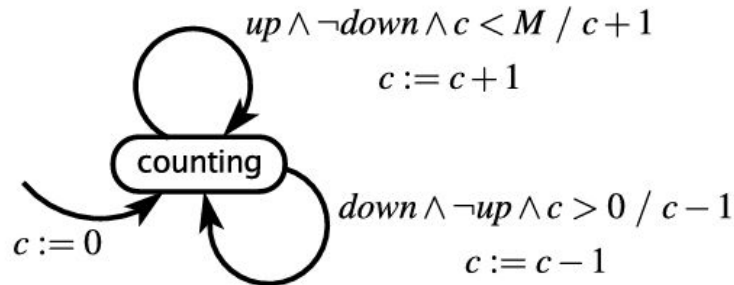
The update function is given by

$$\text{update}(s, i) = \begin{cases} (s + 1, s + 1) & \text{if } s < M \\ & \wedge i(up) = present \\ & \wedge i(down) = absent \\ (s - 1, s - 1) & \text{if } s > 0 \\ & \wedge i(up) = absent \\ & \wedge i(down) = present \\ (s, absent) & \text{otherwise} \end{cases}$$

Extended State Machines

An extended state machine solves this problem by augmenting the FSM model with variables that may be read and written as part of taking a transition between states

variable: $c: \{0, \dots, M\}$
inputs: $up, down$: pure
output: $count: \{0, \dots, M\}$



Nondeterminism

- The system can choose more than one possible next step.
- One state + one input $\rightarrow$ Multiple possible next states
- In a state machine, nondeterminism happens when:
 - From the same state, there are multiple transitions
 - Their conditions (guards) can be true at the same time

Reference:

1. Introduction to Embedded Systems- A Cyber-Physical Systems Approach, Second Edition, E. A. Lee and S. A. Seshia.

Thank You!